

'std::direct_init' for plugging the metaprogramming constructor hole

- Document number: P0271R0
- Date: 2016-02-13
- Authors: Richard Smith, Tony Van Eerd, and David Sankel
- Reply-to: David Sankel david@stellarscience.com
- Audience: Library Evolution

Introduction

There's a hole in our template-metaprogramming facilities. We can figure out at compile time the arguments of any function by name...except for those of a constructor. This seemingly innocuous limitation has a drastic negative impact on `std::variant` in that a `std::variant<int, float>` can surprisingly get into the "corrupted_by_exception" state. There are likely several other instances where this hole causes problems. This paper proposes a modest new standard library type, `std::direct_init` that removes the hole and solves the variant problem.

Variant Problem

The expectation is that it is impossible to get a `std::variant` with "friendly" alternative types, such as `int` and `float`, into the "corrupted_by_exception" state. On the LEWG reflector, however, Augustín K-ballo Bergé demonstrated that a specially constructed class can put any variant into the "corrupted_by_exception" state.

Consider the following code:

```
struct nasty { operator int() const { throw 42; } };
```

```
// elsewhere...
```

```
variant<int, float> v;  
v.emplace<int>(nasty{});
```

Note carefully, that *inside* `emplace` this does something like

```
destroy_old_value();  
new (storage) int(std::forward(arg));
```

ie.

```
destroy_old_value();  
new (storage) int(nasty_arg)
```

And the conversion from `nasty` to `int` happens *inside the* `emplace` function after the old value has been destroyed.

We really want `emplace` to construct in place, not in a temporary followed by a move (otherwise things like `std::mutex` won't work). So we can't create a temporary first. What we would like to do is make temporaries of the args:

```
int && arg0 = int(std::forward(args)); // might throw here  
destroy_old_value();  
new (storage) int(arg0);
```

To do this, we need to know which constructor would be called, and be able to make temporary refs of all the args casted into the constructor args. Unfortunately, there doesn't seem to be a way to do this with C++.

'std::direct_init' to the Rescue

`std::direct_init<T>` represents the direct initialization of a `T` object. For every constructor and constructor template in `T`, `std::direct_init<T>` has a corresponding call operator. The signature of the call operator mimics that of its corresponding constructor while its return type is always `T`. The semantics of the call operator is that it will return a `T` object that is initialized

by the corresponding constructor.

If T is a non-class type we still want `std::direct_init<T>` to model initialization. In the $T(\text{arg})$ case `std::direct_init<T>`'s call operator will have a non-reference parameter type if the cast performs an lvalue-to-rvalue conversion, and otherwise has the appropriate reference type.

Variant Problem Solved

Library Fundamentals Version 2 (N4564) provides `std::raw_invocation_type` which can be used to inspect the parameter types of an invocable function given a sequence of argument types. This tool, when combined with `std::direct_init<T>`, allows `std::variant`'s `emplace` implementation to convert `emplace`'s arguments into the appropriate alternative's parameter types before the constructor is actually called. Consider the following sketch implementation of `emplace`.

```
template<typename T> struct nondeduced { using type = T; };
template<typename T> using nondeduced_t = nondeduced<T>::type;

template<typename X, typename ...T> void variant<...>::emplace_impl(X (*)(T...), nondeduced_t<T> ...t) {
    static_assert(noexcept(X(std::forward<T>(t)...)), "need noexcept constructors"); // or switch to a different technique or wh
    call_dtor(kind, storage);
    kind = kindof<X>;
    new (storage) X(std::forward<T>(t)...);
}

template<typename X, typename ...T> void variant<...>::emplace(T &&...t) {
    emplace_impl((std::raw_invocation_type_t<direct_init<X>(decltype(t)...>*)nullptr, std::forward<T>(t)...));
}
```